PROJECT DOCUMENTATION

LIBRARY MANAGEMENT SYSTEM Using PHP and MySQL

Student name: Aman . B. Prajapati
AF code:AF05331981

Project Type: Web Application
Technology Stack: PHP, MySQL, HTML5, CSS3, Bootstrap, JavaScript
Development Environment: XAMPP / WAMP
Project Level: Beginner to Intermediate
Document Version: 1.0

---

1. Project Overview

1.1 Project Title

Library Management System

1.2 Introduction

The Library Management System is a web-based application developed using PHP, Laravel, and MySQL. It helps a school, college, or small library manage book records, members, book issuing and returning, due dates, fines, and basic reports in one centralized system.

1.3 Problem Statement

Manual library records can be difficult to maintain. Staff may spend time searching registers, tracking issued books, identifying overdue returns, and correcting duplicate or missing entries. This project aims to organize library information digitally and make routine operations easier to track.

1.4 Objectives

- Maintain book and member records in a centralized MySQL database.
- Allow authorized staff to add, view, update, and delete records.
- Record book issue dates, due dates, and return dates.
- Show book availability and identify overdue loans.
- Calculate fines according to configurable library rules.
- Provide search, dashboard summaries, and basic reports.
- Use Laravel features to organize application code and improve maintainability.

1.5 Project Scope

The first version is intended for a small school, college, or community library. It focuses on administration, catalogue management, member management, lending, returns, fines, and reports. Barcode scanning, online payments, SMS alerts, and multi-branch support can be considered future enhancements.

---

2. Functional Requirements

The application should provide the following functionality.

No.	Module	Description
1	Admin / Librarian Login	Authenticate authorized staff and protect administrative pages.
2	Dashboard	Show summary cards such as total titles, copies available, active members, books currently issued, and overdue loans.
3	Book Management	Add, view, edit, delete, and maintain book details such as title, ISBN, publisher, year, and copy count.
4	Category Management	Group books by subjects such as Programming, Science, History, and Literature.
5	Author Management	Maintain author names and related catalogue information.
6	Member / Student Management	Register members and maintain their ID, name, contact details, and email.
7	Book Issue Management	Issue an available copy to a member and save issue date and due date.
8	Book Return Management	Record the return date and update book availability.
9	Fine Management	Calculate and record overdue fines based on the library's configured rate and policy.

10	Search Catalogue	Search by title, author, ISBN, or category.
11	Availability Management	Display available, issued, or unavailable copy status.
12	Reports	Show issue and return history, overdue books, member activity, and inventory summaries.
13	Logout	End the staff session securely.

2.1 Non-Functional Requirements

- Security: use Laravel authentication, authorization checks, CSRF protection, validation, and password hashing.
- Usability: provide a clear interface and readable forms and tables.
- Performance: use database indexes and pagination when the dataset grows.
- Maintainability: separate routes, controllers, models, migrations, and views.
- Reliability: validate inputs and handle database or application errors safely.
- Data integrity: use database constraints and transactions where multiple related changes must succeed together

---

3. Technology Stack

Technology	Purpose
PHP	Server-side programming language for application logic.
Laravel	PHP framework for routing, controllers, models, validation, middleware, migrations, and Blade views.
MySQL	Relational database for books, members, loans, categories, authors, fines, and staff accounts.
HTML5	Page structure.
CSS3	Page styling.
Bootstrap	Responsive layout and interface components.
JavaScript	Client-side interactions such as confirmation dialogs.
Composer	PHP dependency management and Laravel installation.
XAMPP	Local Apache and MySQL environment; configure PHP to a version supported by the chosen Laravel release.
Visual Studio Code	Code editor.

---

4. System Architecture

The application follows a typical Laravel MVC structure:
Presentation Layer — Blade, HTML, CSS, Bootstrap, JavaScript: Displays pages, forms, tables, and validation messages.
Application Layer — Routes, Controllers, Middleware, Services: Receives requests, checks permissions, validates input, and applies library rules.
Model Layer — Eloquent Models: Represents database entities and provides structured database access.

Database Layer — MySQL: Stores catalogue, member, loan, and fine records.

Request Flow

1. The staff member opens a page in the browser.
2. Laravel routes the request to the appropriate controller.
3. Middleware checks authentication and access permissions where required.
4. The controller validates the submitted data and calls the model or business logic.
5. Eloquent reads or updates records in MySQL.
6. Laravel returns a Blade page with the result or a validation message.

---

5. Database Design

5.1 Database Name

library_management

Table	Suggested Columns	Purpose
users	id, name, email, password, role, timestamps	Staff/admin login accounts. Laravel authentication may use this table.
books	id, title, isbn, publisher, publication_year, category_id, author_id, total_copies, available_copies, timestamps	Book catalogue and copy counts.
members	id, member_code, name, email, phone, address, status, timestamps	Student or library member details.
categories	id, name, description, timestamps	Book subject categories.
authors	id, name, biography, timestamps	Author details.
loans	id, book_id, member_id, issued_by, issue_date, due_date, return_date, status, timestamps	Issue and return history.
fines	id, loan_id, amount, paid_amount, status, timestamps	Fine amount and payment status.

5.4 Database Relationships

- A category can contain many books.
- An author can be linked to many books; if books can have multiple authors, use a book_author pivot table.
- A member can have many loan records.
- A book can appear in many loan records over time.
- A loan can have zero or one fine record, depending on the fine policy.

The issued_by field in loans identifies the staff user who processed the issue

5.5 Sample SQL

```
CREATE DATABASE library_management;
USE library_management;

CREATE TABLE categories (
     id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
     name VARCHAR(100) NOT NULL UNIQUE,
    description VARCHAR(255) NULL,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL
);

CREATE TABLE authors (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    biography TEXT NULL,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL
);

CREATE TABLE books (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    isbn VARCHAR(30) NULL UNIQUE,
    publisher VARCHAR(150) NULL,
    publication_year YEAR NULL,
    category_id BIGINT UNSIGNED NULL,
    author_id BIGINT UNSIGNED NULL,
    total_copies INT UNSIGNED NOT NULL DEFAULT 1,
    available_copies INT UNSIGNED NOT NULL DEFAULT 1,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL,
    FOREIGN KEY (category_id) REFERENCES categories(id),
    FOREIGN KEY (author_id) REFERENCES authors(id)
);

CREATE TABLE members (
```

6. Module Description

6.1 Admin Authentication

Staff log in using registered credentials. Laravel authentication, password hashing, middleware, and authorization policies should protect administrative actions.

6.2 Dashboard

Displays summary counts for books, available copies, members, active loans, and overdue loans. Counts should be calculated from the database.

6.3 Book Management

Staff can create, view, edit, and delete book catalogue records. Validate required fields and prevent deletion where active loan records would be affected.

6.4 Member Management

Staff can register members, update contact details, view member histories, and deactivate a member when necessary.

6.5 Issue Book

The system checks that a member is active and a copy is available, then records issue and due dates. Use a database transaction to keep loan records and copy counts consistent.

6.6 Return Book

Staff locate the active loan, record the return date, and increase available copies. Prevent the same loan from being returned twice.

6.7 Fine Management

Calculate late days from the due date and return date, or the current date for an overdue loan. Apply a clearly defined per-day rate and record payments. The rate is a library policy and should be configurable.

6.8 Search Catalogue

Search by title, ISBN, author, or category. Use validation and pagination for larger catalogues.

6.9 Reports

Generate reports for issued books, returned books, overdue loans, unpaid fines, and inventory. Export options can be added later.

6.10 Logout

End the authenticated session and redirect the staff member to the login page.

---

7. Suggested Project Folder Structure

```
library-management/
├── app/
│   ├── Http/Controllers/
│   │   ├── DashboardController.php
│   │   ├── BookController.php
│   │   ├── MemberController.php
│   │   ├── LoanController.php
│   │   └── FineController.php
│   └── Models/
│       ├── Book.php
│       ├── Member.php
│       ├── Category.php
│       ├── Author.php
│       ├── Loan.php
│       └── Fine.php
├── database/
│   ├── migrations/
│   └── seeders/
├── resources/views/
│   ├── layouts/
│   ├── dashboard.blade.php
│   ├── books/
│   ├── members/
│   ├── loans/
│   └── fines/
├── routes/
│   └── web.php
├── .env
└── composer.json
```

This is an illustrative structure; exact files depend on the Laravel version and installed authentication starter kit.

8. Development Methodology

The project follows a simple sequential development process.

1. Requirement Analysis: decide user roles, lending limits, loan duration, and fine policy.
2. System Design: plan pages, workflows, tables, and relationships.
3. Laravel Setup: create the project with Composer and configure the environment.
4. Database Setup: create migrations and run them against MySQL.
5. Model and Controller Development: define Eloquent relationships, validation, and business rules.
6. Interface Development: build responsive Blade views using Bootstrap.
7. Security and Validation: protect routes, validate input, and check permissions server-side.
8. Testing: test each module and the relationships between modules.
9. Deployment: configure production environment variables and deploy to a suitable server.
10. **Deployment:** Configure the application on a suitable web server.

---

9. Validation and Exception Handling

- Validate all form input on the server using Laravel validation rules.
- Use Laravel's CSRF protection for state-changing web forms.
- Use Eloquent or parameterized queries; do not concatenate user input into SQL.
- Hash passwords; never store plain-text passwords.
- Protect routes with authentication middleware and check roles/permissions for sensitive actions.
- Escape user-provided content in views to reduce cross-site scripting risk.
- Use database transactions for issuing and returning books when multiple records are changed.
- Keep secrets in environment configuration and do not commit real credentials to version control.
- Log technical errors securely and show users clear, non-sensitive messages.

---

10. Testing

Testing verifies that the application behaves as expected.

10.1 Test Cases

Test ID	Test Scenario	Expected Result
TC01	Login with valid credentials	Authorized dashboard is shown.
TC02	Login with invalid credentials	Access is denied and a safe message appears.
TC03	Add a book with valid details	Book record is saved.
TC04	Submit missing required book fields	Validation errors are shown.
TC05	Register a member with duplicate member code	Duplicate is rejected.
TC06	Issue an available book	Loan is created and available copies decrease.
TC07	Issue a book when none is available	Issue is blocked.
TC08	Return an active loan	Return date is saved and availability is updated.
TC09	Return the same loan twice	Second return is rejected.
TC10	Open an admin page without logging in	User is redirected or access is denied.
TC11	Search by book title or ISBN	Matching books are displayed.
TC12	Check overdue loan	Overdue status and applicable fine are calculated.

10.2 Types of Testing

- **Unit Testing:** Tests individual functions and components.
- **Integration Testing:** Tests PHP-to-database interactions.
- **Functional Testing:** Verifies that each feature meets its requirements.
- **Security Testing:** Checks authentication, authorization, input handling, and SQL injection protection.
- **User Acceptance Testing:** Confirms that the application meets user expectations.

---

11. Installation and Execution

Prerequisites

- PHP version compatible with the selected Laravel release.
- Composer installed.

- MySQL server running through XAMPP or another supported setup.
- Visual Studio Code or another code editor.
- A web browser.

Steps

1. Install PHP and Composer, then confirm both work in the terminal.
2. Create a Laravel project using Composer (for example, composer create-project laravel/laravel library-management).
3. Create a MySQL database named library_management using phpMyAdmin or MySQL tools.
4. Update the database connection settings in the project's .env file: DB_DATABASE=library_management, DB_USERNAME and DB_PASSWORD according to the local MySQL configuration.
5. Create the required Laravel migrations, models, controllers, and Blade views.
6. Run php artisan migrate to create the tables defined by the migrations.
7. Configure an authentication starter kit or implement secure Laravel authentication before using protected pages.
8. Start the development server with php artisan serve.
9. Open the local address shown in the terminal, commonly http://127.0.0.1:8000, and test each module.

Important: The commands above describe the setup process. The project must have its migrations, routes, views, and authentication configured before all features will work. Do not assume the database schema in this document has been installed automatically.

---

12. Limitations

- The initial scope supports a single library location.
- Barcode/RFID scanning is not included.
- Online fine payment and payment gateway integration are not included.
- Email/SMS reminders are not included in the basic version.
- The exact fine rules and borrowing limits must be defined by the library.

---

13. Future Enhancements

The following features can be added:

1. Barcode or QR code scanning for books.
2. Email reminders before and after due dates.
3. Member self-service login and book reservation.
4. PDF and Excel report export.
5. Online fine payment integration.

6. Multiple library branches and inventory transfers.
7. Dashboard charts and activity logs.
8. REST API for a mobile application.

---

14. Conclusion

The Library Management System demonstrates how PHP, Laravel, and MySQL can be combined to build a structured web application for managing library resources. The project covers catalogue management, member records, lending, returns, overdue fines, authentication, validation, and reporting. Laravel provides an organized MVC structure, while MySQL stores the application's records. The implementation should be tested and adapted to the actual library's rules.

---

15. References

1. PHP Official Documentation — PHP Documentation
2. Laravel Official Documentation — Laravel Documentation
3. MySQL Official Documentation — MySQL Documentation
4. Bootstrap Official Documentation — Bootstrap Documentation
5. MDN Web Docs — MDN Web Docs
6. OWASP Web Application Security Guidance — OWASP

This documentation is intended for learning and reference purposes. Before submitting the project academically or professionally, update the requirements, implementation details, screenshots, database schema, test results, and references to reflect the actual implementation.